Executable Conformance for a Secure-Multiparty Application: An MP-SPDZ Case Study on What Implementation-Level Evidence Checks, and Where It Stops Short of a Privacy Proof

Experience report / case study — artifact baseline commit `305a3a8`

Harshit Kargeti*

August 3, 2026

Abstract

This is a single-application *case study*. We report what a stack of lightweight, testable checks does and does not establish for one secure-multiparty computation (MPC) application: the deterministic-query fragment of the knowledge-threshold belief computation of Mardziel, Hicks, Katz and Srivatsa (PLAS 2012), implemented in MP-SPDZ (three-party replicated secret sharing, Rep3). The checks are a separately implemented plaintext *pseudo-oracle* derived from the same paper, an anti-echo interface that keeps expected outputs out of the circuit, a strict runtime transcript check, and recomputed, hash-bound evidence. On the tested inputs these provide useful, coverage-bounded assurance; we are careful to state each guarantee operationally rather than as formal soundness. We then run an *adaptive, retrospectively discovered* mutation sequence against a static delivery linter under a precisely bounded build-time adversary that controls the application source but not the oracle, checker, policy, CI workflow, evidence validator, or pinned compiler. The sequence exposes the limit of the studied linter: a masked comparison-open and a raw secret-reveal are the same opcode, and the MP-SPDZ `call_tape`/`call_arg` register channel that a leak could use is the same channel the honest comparison subtapes use, so it cannot be blocklisted without rejecting the honest build. We conclude only that *this opcode-identity / channel-blocklist linter cannot be promoted to a general non-leakage checker*; distinguishing legitimate from leaking flows needs analysis stronger than opcode/channel identity (operand/provenance-sensitive dataflow, typing, or a verified IR), which we did not evaluate. We do not claim this principle is new — that leakage is a dataflow rather than opcode-identity property is established for constant-time and smart-contract bytecode analysis; our contribution is the MPC-application instantiation, the pinned-backend mechanism, and a reproducible mutation study. We archive the decisive mutations and a machine-readable detection matrix, and we are explicit about adaptive overfitting, the trusted-computing-base boundary, and the absence of independent validation.

*Author list, affiliations, and acknowledgments are **[OPEN: author metadata]**. **Correlated-authorship disclosure:** the circuit, the pseudo-oracle, the tests, the review synthesis, and this manuscript were produced within a single project and substantially with AI assistance; model agreement is not independent validation (see §8).

1 Introduction

An MPC framework such as MP-SPDZ [2] compiles a program into a protocol that several parties execute so each learns only its prescribed output. Building a correct *application* on top of such a framework is separate from the framework's own security argument: an application author can, by accident or intent, write source that reveals a value the functionality was meant to protect. When such an application is wired into continuous integration (CI), a green check is easy to mistake for evidence that the privacy property holds. This paper is a case study of what that check can and cannot establish.

We are explicit about what this paper is *not*. It is not "the first implementation of PLAS belief tracking," and makes no such claim: earlier adversarial review on this project established, correctly, that implementing and timing a 2012 construction is not by itself a research contribution [docs/review-log.md R-10]. It is also not a new analysis *technique*: the principle that information-flow safety depends on dataflow and semantics rather than instruction names is prior art (§7). We frame the work as an *experience report / case study* whose value is the concrete instantiation, the reproducible mutation study, and the honest accounting of the assurance boundary.

A narrowed thesis. On one MP-SPDZ application, separately implemented functional pseudo-oracles, anti-echo interfaces, transcript checks, and recomputed evidence provide useful coverage-bounded assurance; an adaptive mutation sequence exposes the limit of the studied opcode-identity delivery linter, including the MP-SPDZ `call_tape`/`call_arg` path, under a precisely bounded build-time adversary.

Contributions.

1. An **application** of established conformance / pseudo-oracle practice to an MPC application (§3), with the anti-echo interface and the recomputed evidence discipline as the local delta.
2. A **reproducible case-study artifact** for the deterministic-query fragment of `threshold_SMC` (§3.1): a fixture and a bounded 228-case adversarial matrix checked against the pseudo-oracle, and five committed executable negative controls, against a pinned backend.
3. A **reproducible adversarial mutation study** (§4): one motivating transcript failure, six compiled-delivery mutation cases, and one semantic mutation, with an archived corpus and a machine-readable detection matrix, showing the limit of the studied linter *at its exact strength*.
4. A statement of the **assurance boundary and hand-off** (§5): the operational property the checks demonstrate, and the two protocol-level properties a real non-leakage guarantee would additionally need.

We report the three-way adversarial review process (§6) as an *experience lesson*, not a scientific contribution: we have no comparator or controlled evaluation and do not claim it outperforms a single review pass.

2 Background and two adversaries

The functionality (deterministic-query fragment). Following PLAS 2012 [1], N parties hold secrets in a finite domain D; each observer j holds a belief over the others' secrets. For a query Q, the mechanism checks, per recipient, whether releasing Q's output would let any protected party's posterior exceed a public threshold, quantifying over *every possible output* so the

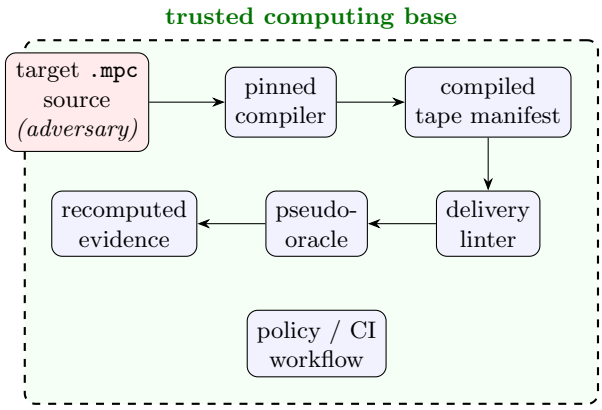

Figure 1: Assurance-boundary architecture. The build-time adversary controls only the target source (red); everything inside the dashed box is trusted. The runtime Rep3 adversary (not shown) is a separate model addressed only by the protocol-level hand-off of §5.

accept/reject decision is simulatable; on accept the recipient privately learns the output, on reject a masked constant, and no party may learn another's verdict [conformance/CONTRACT.md]. We study deterministic, total, public queries; two instantiate it: `p1_is_max` and `sum_even`.

Backend. Circuits are compiled and run with MP-SPDZ pinned at `9d809599`, backend `replicated-ring-party.` three-party replicated secret sharing over $\mathbb{Z}_{2^{64}}$, semi-honest, honest majority. The compiler emits a manifest of *tapes* (a main tape plus comparison subtapes). Two facts about this compiled form recur. First, a public reveal and a legitimate masked comparison-open are the *same* opcode: MP-SPDZ "uses a single instruction for opening" [2], and the correctness-check flag does not change this [conformance/delivery_inspect.py]. Second, a value in one tape reaches another only through memory or through register arguments passed by `call_tape` (opcode `0xec`) and received by `call_arg` (opcode `0xed`); a cross-tape register reference is a compiler error [MP-SPDZ docs: instructions.html].

Two separate adversaries. We keep two adversaries distinct throughout; conflating them is what makes "source-controlling author" sound vacuous.

Runtime Rep3 adversary. At most one semi-honest corrupted party among the three; honest majority; no collusion; authenticated encrypted channels; process isolation (stated, *not* enforced by the single-host test harness). This is the adversary a *protocol-level* privacy argument would target. Our functional and linting evidence does *not* address it; that hand-off is §5.

Build-time mutation adversary. Controls the target `.mpc` application source, *and only that.* The pseudo-oracle, the linter/checker, the policy (thresholds, expected delivery), the CI workflow, the evidence validator, and the *pinned* MP-SPDZ compiler are all in the trusted computing base (TCB) and are *not* under adversary control. The mutation study (§4) is conducted against this adversary. Were the TCB itself adversarial, every automated check would be trivially void; stating the boundary is what makes the study meaningful.

Figure 1 draws this boundary. We make no claim under collusion, a malicious party, unencrypted channels, or a shared host, and no performance claim.

3 Method: conformance checks for one MPC application

Each element exists to defeat a specific way a conformance check can be vacuous. We state what each element *operationally* provides, not that it is "sound."

A separately implemented pseudo-oracle. The plaintext oracle is written from the paper, separately from the circuit and from the project's earlier exact-rational reference model, and transcribes the all-possible-outputs check directly [conformance/oracle.py]. It is a *pseudo-oracle* (a separate implementation of the same specification), *not* an independently validated ground truth: it shares the specification with the circuit, so a *correlated* misreading of PLAS could make both agree on the wrong behaviour. It did carry one fatal transcription error (conditioning on the actual output only) until review caught it [docs/review-log.md R-13]. We state this shared-spec / correlated-error risk wherever we rely on oracle agreement (§8).

An anti-echo interface. Before any circuit was written, the interface was fixed so that expected outputs and expected post-state *never enter the circuit* — neither compiled in nor as runtime inputs; they live only in the external harness, obtained from the pseudo-oracle [conformance/INTERFACE.md; R-17,R-18]. Circuit inputs are exactly the public parameters and the secret-shared secrets and beliefs, and the harness runs at least one state beyond the fixture, so a constant-output circuit cannot pass by accident. Beliefs travel as dense unnormalized non-negative integer weights; the threshold check avoids division ($b\,M \le a\,Z$). Because MP-SPDZ comparison is signed, the no-wraparound requirement is $B < 2^{k-1}$ (not the naive $2^k > B$), which we pin with a counterexample [circuit_spec.py; R-22].

A strict transcript check. A correct party's captured stdout must be exactly its two own records; any foreign, duplicate, unknown, missing, or extra line makes the check *fail closed* [conformance/private_run.py] This is an operational property over the enumerated transcript cases on the observed stdout channel; it observes selected output channels and does not establish absence of every possible leak.

Recomputed, hash-bound evidence. Each run retains a typed record; in CI the validator requires each record's source hash and delivery signature to equal values recomputed from the checked-out source and a fresh compile, and binds each record to a canonical case via a recomputed input hash and pseudo-oracle verdict [conformance/validate_evidence.py; R-34,R-37,R-43]. This establishes *self-consistency and provenance* of the evidence *under the trusted CI workflow*; it is not integrity against an adversary who can alter source, checker, workflow, and evidence together (that adversary is outside the TCB of §2). A `pipefail` canary prevents a failing suite from passing through a logging pipe [R-31].

3.1 Artifact and results

Foregrounding properties and inputs rather than counts: on the fixture (two invocations: one all-accept, one with a per-recipient reject and belief preservation) plus two further valid states, the circuit reproduces the pseudo-oracle's visible outputs, payloads, and reconstructed weights. A *bounded, constructed* adversarial matrix of 228 cases (216 single-invocation + 12 carried; all 27 secrets × two queries × uniform / non-uniform / per-party-scaled integer weights near the bit bound) all match the pseudo-oracle [coverage.py; results-coverage.txt]. This is a constructed test matrix, *not* general coverage. The conformance package holds 92 tests — 46 functional and 46

delivery-gate/linter tests — and 9 reference tests, all green; test counts are not themselves a result. Five committed executable negative controls are rejected by the linter (§4). No timing is reported: the repository's older timings measure a different, non-conforming circuit and must not be cited [README; docs/gap.md].

4 The adversarial mutation study

Functional conformance says nothing about *how* a value is delivered: a circuit can compute correct verdicts and broadcast them. We therefore built a static compiled-delivery linter and subjected it to an *adaptive* sequence of source mutations by the build-time adversary. We are explicit that this is a *retrospective, adaptively discovered* sequence, not a complete or independently designed taxonomy: the checker was repeatedly hardened against the same mutations later used as evidence (adaptive overfitting), and there are no held-out mutations from which to estimate detector effectiveness (§8).

Accounting. One motivating transcript failure (**B0**); six compiled-delivery mutation cases (**B1**–**B6**); one semantic mutation (**S1**). The six are not homogeneous: B1, B2, B3 and B5 are committed executable source controls; **B4 is a synthetic compiled-manifest counterexample, not a source** `.mpc`; and B6 is a reconstructed source mutation whose naive realization is itself caught (§4.1). B0: changing `reveal_to(j)` to a public `reveal()` while keeping the per-party print compiles to a public open of every verdict, yet a stdout-only checker reported success [R-41]; that motivated inspecting the *compiled* delivery. B1–B3 and B5 then each defeated the then-current linter *as committed source builds* and were closed by adding a rule (separate-tape reveal; name-spoofed subtape; `reveal(False)` — still a public open; and the vectorized-memory variant). **B4** is different: it is the rr-5 observation that the multiset rule is blind to tape *bodies* — a `reveal(False)` of the verdict placed in an *existing* `EQZ(3)_63` body with the three pinned names intact. R-45 demonstrated this with a *synthetic manifest*, not a compiled source mutation; against the final linter it is rejected by the subtape-memory rule (the multiset is left intact), which is why those rules were added [review-log R-42..R-45]. The clean build stores nothing in its main tape and touches no memory in its subtapes, so forbidding memory is a restriction with no false positives on the honest build.

4.1 B6, at its exact strength

We scope B6 precisely, because it is easy to overstate. We reconstructed a mutation that routes party 1's verdict through the `call_tape`/`call_arg` register channel (a `@function_tape` that receives the verdict as a register argument and opens it). Compiled against the pinned backend and run through the final linter, this mutation is **REJECTED**: the added tape breaks the pinned comparison-subtape multiset, and MP-SPDZ's call frame also spills through memory, tripping the memory rules. So **B6 is not a demonstrated bypass of the final linter.**

Its content is narrower and precise. The *honest* build passes secret comparison operands across the tape boundary via `call_tape` (in main; e.g. `call_tape 12, 1, ci0, ...s202(81), s0(81) ...`) received by `call_arg` in each of the three comparison subtapes — no memory (reproducible; `paper/mutations/`). Therefore a blocklist rule that forbade cross-tape register-argument transfer of secrets, the way rules against memory close the memory channel, would **reject the honest build**. The `call_arg` channel *cannot be closed by an opcode/channel blocklist* without rejecting the honest build. Combined with the opcode-identity fact (a masked open and a raw reveal are the same instruction), the lesson is precise: **the studied opcode-identity / channel-blocklist linter cannot be promoted to a general non-leakage checker.** `call_tape`/`call_arg` cannot simply be blocklisted, because the honest build uses that channel. Distinguishing legitimate

ID	Committed?	Mutation / channel	Final-linter outcome	Notes
B0	`leaky`	public `reveal()` in main	REJECTED	motivating stdout failure
B1	`subleak`	separate `@function_tape` reveal+print	REJECTED	closed by whole-manifest inspection
B2	`namespoof`	name-spoofed subtape: wrong-player delivery + file sink	REJECTED	closed by content rules
B3	`openfalse`	`reveal(False)` in spoofed `EQZ(3)_63`, scalar mem	REJECTED	`False` open is still public
B4	*synthetic*	`reveal(False)` in an *existing* `EQZ(3)_63` body, multiset intact	REJECTED (rule h)	synthetic manifest (R-45); **not a source `.mpc`**
B5	`openfalse_vec`	same via vectorized `vstms/vldms`	REJECTED	closed by broadened regex
B6	*reconstructed*	verdict via `call_tape/call_arg` register args	REJECTED (naive)	see §4.1; not a bypass of the final linter
S1	*reconstructed*	correct delivery, reject-mask dropped	PASS linter / **FAIL oracle**	semantic; linter is semantics-blind

Table 1: Detection matrix (machine-readable form: `paper/mutations/detection_matrix.json`; outcomes recomputed against `delivery_inspect.py`@`305a3a8`, MP-SPDZ `9d809599`). The five committed controls map to B0–B3 and B5 (`leaky, subleak, namespoof, openfalse, openfalse_vec`); they do *not* map one-to-one to every row. **B4 is a synthetic compiled-manifest counterexample (R-45), not a source `.mpc`**; B6 and S1 are reconstructed source mutations under `paper/mutations/`. Not committed to the frozen artifact: B4, B6, S1.

from leaking flows requires analysis stronger than opcode/channel identity — operand/provenance-sensitive dataflow, typing, a verified IR, or other semantic machinery — *which we did not evaluate* (§7). We also do *not* claim that memory and register arguments are the *only* cross-tape channels; the demonstrated `call_arg` path is sufficient for this scoped conclusion.

4.2 S1: correct recipient, wrong value

On a different axis, a mutation that drops the reject-mask (delivers the true output even to rejected recipients) **passes** every delivery-linter rule — the delivery structure is intact — yet **fails** the pseudo-oracle: on secrets $= (0, 0, 0)$, `p1_is_max`, the true output is 1 and all parties reject (oracle payload 0), but S1 delivers 1 to all three (`paper/mutations/threshold_smc_wrongsem.mpc`). We ran this **end-to-end at the pinned backend**: all three parties complete (return code 0) and each delivers `ACCEPT 0 / PAYLOAD 1`, an executed oracle mismatch at parties $[0, 1, 2]$ (raw per-party stdout, oracle-vs-delivered verdicts, source hash, and compile command archived in `paper/mutations/s1_runtime_evidence.json`). Delivery-linting checks *how* values are delivered, not *which*; only the functional pseudo-oracle, bounded by test coverage, constrains that.

5 The assurance boundary

Separating three levels keeps the claims honest.

1. **General prior principle (not novel here):** information-flow safety depends on dataflow and

semantics, not opcode names. Established for constant-time and smart-contract bytecode (§7).

2. **Case-study lesson (transferable, not empirically validated across applications):** adaptive linting of a security property can produce false confidence, and such tooling should print its assurance boundary; ours does [delivery_inspect.py banner].
3. **Pinned-backend finding (backend/version specific):** MP-SPDZ's shared open instruction and the `call_tape`/`call_arg` path at `9d809599`, and the consequence that the studied opcode-identity / channel-blocklist linter cannot be promoted to a general non-leakage checker (`call_tape`/`call_arg` cannot be blocklisted, since the honest build uses it).

What would make it a real guarantee. Two protocol-level properties, neither supplied by our checks: (i) *privacy of the executed circuit* — a Rep3 simulation argument (a human MPC specialist) or a formally-verified, dataflow-checked comparison primitive; and (ii) *semantic source-to-spec binding* — that the circuit computes the intended function, not merely one that agrees with the pseudo-oracle on tested cases (S1 shows why privacy of delivery is insufficient without it). A Rep3 proof establishes (i) and not (ii); our checks give coverage-bounded evidence toward (ii) and nothing toward (i).

6 The adversarial review process (experience)

The results were produced by a three-party loop — direction, implementation, and adversarial review — with a shared repository as the only mailbox [docs/workflow.md], and an append-only 47-entry review log. We report this as an *experience lesson*, not a scientific result: we have no single-pass comparator, no ablation, and no independent replication, so we cannot claim reproduce-before-fix review *outperforms* a single pass. Descriptively, iterative adversarial review surfaced a fatal specification error (§3) and the entire mutation sequence, and it recorded its own missteps (a soundness premise asserted in one round was refuted in the next). Whether this generalizes is unestablished.

7 Related work

Novelty statements below rest on a bounded, documented search [paper/LITERATURE_REVIEW.md]; they are "to the best of a bounded search," not absolute (§8).

Leakage is dataflow, not opcode identity (the prior principle). ct-verif reduces constant-time to a 2-safety property on LLVM IR [3]; Binsec/Rel decides constant-time *relationally at the binary/opcode level* on operand provenance, showing compiler backends inject leaks invisible to instruction-name checks [4]; Securify decides EVM-bytecode security on inferred *dataflow* facts rather than syntactic patterns [5]. Our §4.1 instantiates this principle in the MPC-application setting; we claim the instantiation and the pinned-backend mechanism, not the principle.

Language-based security for MPC (the positive dual). Kerschbaum's information-flow type system certifies non-leakage of a mixed-protocol secure computation at the *source* level with a trusted compiler [6]. Closest to our finding, Skalka and Near give a *sound* static security-type system for a purpose-built low-level MPC IR (Overture/Prelude), proving information is released only through explicit declassifications [7, 8]. It is the positive dual of our negative case-study observation: it is sound *because* its IR makes declassification syntactically distinct — the property

MP-SPDZ bytecode lacks — and it assumes a corrupt *party*, not a malicious *author*. Verified MPC stacks and mechanized simulation proofs (Wys*, verified SFE, EasyUC, replicated-secret-sharing proofs) are the protocol-level targets our hand-off (§5) defers to [LITERATURE_REVIEW.md §3].

Conformance / differential testing and negative controls. Independent and pseudo-oracle testing is standard [9]; Wycheproof applies reference-oracle conformance to crypto *primitives* (feeding expected outputs, the opposite of our anti-echo design) [LITERATURE_REVIEW.md §4]; MASC evaluates crypto-misuse detectors with planted leaky mutants [10], the closest precedent for negative controls, but as an offline third-party-detector benchmark rather than committed CI controls. Reproducible Builds [11] and in-toto/SLSA are the evidence-discipline neighbours; ours recomputes per-case, oracle-bound evidence rather than attesting whole artifacts.

8 Threats to validity

- **Build-time adversary / TCB.** All results assume the TCB of §2 (oracle, checker, policy, CI, validator, pinned compiler trusted); a repository-controlling adversary voids them. Workflow hashes establish *self-consistency*, not authenticity against such an adversary.
- **Correlated authorship and AI assistance.** Circuit, pseudo-oracle, tests, review synthesis, and manuscript were produced within one project and substantially with AI assistance; model agreement is not independent validation.
- **Adaptive overfitting.** The linter was hardened against the same mutations later used as evidence; there are no held-out mutations, so we do not estimate detector effectiveness.
- **Pseudo-oracle pseudo-independence.** The oracle shares PLAS with the circuit; a correlated spec misreading is possible. A cross-check against the project's exact reference model is limited because that model implements a *different* (single-observer, stateless) functionality [docs/review-log.md R-11]; we therefore state the risk rather than claim it eliminated.
- **Bounded literature search.** IACR ePrint full text and some indexes were not reachable; novelty is claimed only to the best of a documented bounded search [LITERATURE_REVIEW.md §11-12].
- **No independent validation.** No human MPC specialist has signed off and there is no external replication.
- **Construct / external validity.** "Conformance" means agreement with a pseudo-oracle on tested inputs. Results are for one deterministic-query functionality, $N{=}3$, $D{=}\{0,1,2\}$, one backend, and a single-host harness that is not an adversarially isolated deployment; we do not imply an evaluated methodology across MPC applications plural.
- **No performance or simulation-security claim**; the boundary result is a case-study observation, not a theorem.

9 Limitations and future work

Persistent secret-shared state across invocations (Σ_T) is unimplemented and out of scope. The experiment most likely to change a conclusion is a genuine *operand-sensitive dataflow* delivery checker: if it distinguishes the honest `call_arg` flow from a leaking one, it changes the boundary; if it fails, it shows precisely where dataflow is required. A general impossibility statement for

dataflow-free syntactic checkers is plausible but, as we found, much of its natural formulation is definitional and it requires an operational leak definition and a proven safe/leaky pair; we leave it as a future-work sketch rather than a theorem. Cross-application and cross-backend replication would move the case-study lesson toward an evaluated one.

10 Conclusion

On one MP-SPDZ application, a separately implemented pseudo-oracle, an anti-echo interface, a strict transcript check, and recomputed evidence give real, coverage-bounded, operationally-stated assurance and catch a family of gross delivery mistakes — and they have a sharp, nameable boundary. The studied opcode-identity / channel-blocklist linter cannot be promoted to a general non-leakage checker: a masked open and a raw reveal are the same instruction, and the register channel a leak would use is the one the honest code uses, so `call_tape`/`call_arg` cannot be blocklisted. Distinguishing the flows needs analysis stronger than opcode/channel identity (operand/provenance-sensitive dataflow, typing, or a verified IR) — which we did not evaluate — and a privacy guarantee needs a protocol-level proof. Naming that boundary — and printing it next to the green check — is the case study's point.

A Detection matrix (machine-readable)

The archived corpus and machine-readable matrix are `paper/mutations/`: sources `threshold_smc_callarg.mpc` (B6) and `threshold_smc_wrongsem.mpc` (S1); `detection_matrix.json` with, per row, the mutation, the checker version defeated, source/patch, compile command, the `manifest_signature` assembly hash, the linter/runtime/oracle outcome, the committed/reconstructed status, and the evidence location; and `README.md` with reproduction. B0–B3 and B5 reference the five committed controls under `conformance/mpc/` (frozen at `305a3a8`); B4 is a synthetic compiled-manifest counterexample (not committed); B6 and S1 are reconstructed source mutations under `paper/mutations/`.

References

[1] P. Mardziel, M. Hicks, J. Katz, M. Srivatsa. Knowledge-Oriented Secure Multiparty Computation. PLAS, 2012.

[2] M. Keller. MP-SPDZ: A Versatile Framework for Multi-Party Computation. ACM CCS, 2020. IACR ePrint 2020/521. **[OPEN: confirm pages]**

[3] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, M. Emmi. Verifying Constant-Time Implementations. USENIX Security, 2016.

[4] L.-A. Daniel, S. Bardin, T. Rezk. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary Level. IEEE S&P, 2020.

[5] P. Tsankov et al. Securify: Practical Security Analysis of Smart Contracts. ACM CCS, 2018.

[6] F. Kerschbaum. An Information-Flow Type-System for Mixed Protocol Secure Computation. ACM ASIACCS, 2013.

[7] C. Skalka, J. P. Near. Language-Based Security for Low-Level MPC. PPDP, 2024. arXiv:2407.16504.

[8] C. Skalka, J. P. Near. SMT-Boosted Security Types for Low-Level MPC. FASE/ETAPS, 2025. arXiv:2501.17824.

[9] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo. The Oracle Problem in Software Testing: A Survey. IEEE TSE, 2015.

[10] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshyvanyk, A. Nadkarni. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques. IEEE S&P, 2022.

[11] C. Lamb, S. Zacchiroli. Reproducible Builds: Increasing the Integrity of Software Supply Chains. IEEE Software, 2022.